

앱 빌드 · 배포 명세서

shoppinglog.store — Expo (React Native) WebView + 푸시 알림 앱

iOS TestFlight · Android Play Console · EAS Build

대상: 빌드·스토어 배포 담당자

프론트엔드(WebView + 푸시 알림 프론트)는 구현 완료 상태입니다.

이 문서대로 본인 소유 계정으로 빌드 → 테스트 → 배포를 진행하세요.

목차

1. 이 프로젝트 개요 & 담당 범위
2. 준비물 & 사전 설치
3. 계정 3종 · 비용 · 본인인증 (가장 중요)
4. 프로젝트를 내 Expo 계정에 연결 (eas init)
5. 푸시 자격증명 설정 (iOS APNs / Android FCM)
6. EAS 빌드 — APK / AAB / 개발빌드
7. iOS — App Store Connect & TestFlight
8. Android — Play Console 내부 테스트
9. 푸시 알림 실제 동작 테스트
10. 자주 막히는 부분 (트러블슈팅)
11. 명령어 치트시트

1. 이 프로젝트 개요 & 담당 범위

이 앱은 Expo (React Native) 기반 WebView 앱입니다. `https://shoppinglog.store` 를 앱 안에 풀스크린으로 띄우고, Expo 푸시 알림을 받습니다.

구분	내용	상태
프론트엔드	WebView 화면, 푸시 권한·토큰 발급, 알림 탭 시 이동 등 프론트 코드	✅ 완료
백엔드	토큰 저장 + Expo Push API 발송 — 별도 <code>docs/BACKEND.md</code> 참고	백엔드 담당
빌드·배포	EAS 빌드, 스토어 등록, 테스트 (이 문서)	👉 담당자 진행

중요 — 계정 소유권. 이 프로젝트는 아직 어떤 Expo 계정도 연결돼 있지 않습니다(`app.json` 에 `projectId` 없음). 반드시 본인(배포 담당) 소유의 Expo / Apple / Google 계정으로 진행하세요. 아래 4번에서 `eas init` 을 본인 계정으로 실행하면 그때 연결됩니다.

프로젝트 구조 (핵심 파일)

파일	역할
<code>app.json</code>	앱 이름/번들ID/플러그인 설정. 번들ID = <code>store.shoppinglog.app</code>
<code>eas.json</code>	EAS 빌드 프로파일 (development / preview=APK / production=AAB)
<code>src/app/index.tsx</code>	WebView 화면 + 알림 탭 처리
<code>src/lib/notifications.ts</code>	푸시 권한·토큰·채널 설정
<code>docs/BACKEND.md</code>	백엔드 담당자용 연동 문서

2. 준비물 & 사전 설치

항목	설명
Node.js 20+	<code>node -v</code> 로 확인. 없으면 nodejs.org 에서 설치
EAS CLI	<code>npm install -g eas-cli</code>
Expo 계정	무료. expo.dev 가입
실제 기기	푸시 토큰은 실기기 에서만 발급됨 (시뮬레이터 불가)
 Mac	iOS 빌드 자체는 EAS 클라우드에서 됨. 단, TestFlight 관리는 어디서나 가능

```
# 의존성 설치 + 프로젝트 정상 동작 점검
npm install
npx tsc --noEmit      # 타입 체크 (통과해야 함)
npx expo lint         # 린트 (통과해야 함)
npx expo start        # 개발 중 미리보기 (Expo Go 는 푸시 미지원)
```

3. 계정 3종 · 비용 · 본인인증 (가장 중요)

실제 기기에 앱을 설치하고 푸시를 받으려면 아래 3개 계정이 필요합니다. **비용과 본인인증 절차**를 먼저 파악하세요. (인증 승인까지 시간이 걸려서 가장 먼저 시작해야 함)

계정	비용	본인/사업자 인증	승인 소요
Expo	무료	이메일 가입만	즉시
Apple Developer Program 	\$99 / 년	Apple ID + 2단계 인증, 결제. 법인은 D-U-N-S 번호 필요	개인 보통 24~48시간
Google Play Console 	\$25 (1회, 평생)	Google 계정 + 신분증 본인확인 , 주소/전화. 조직은 사업자 정보 + D-U-N-S	본인확인 며칠 걸릴 수 있음

2024년 이후 신규 정책. Google Play 신규 개인 개발자 계정은 **신분증 기반 본인확인**이 필수이고, 일부는 **출시 전 테스터 확보 요건**(예: 비공개 테스트 일정 기간)이 요구될 수 있습니다. Apple/Google 모두 **주소지·신분증(또는 사업자등록)** 인증이 필요하니, 결제 수단과 신분증을 미리 준비하세요.

법인 vs 개인

- **개인 개발자로 등록:** 절차 간단, 신분증 본인확인. 스토어에 개인 실명/닉네임 노출.
- **법인(사업자)로 등록:** **D-U-N-S 번호**(무료 발급, 시간 소요)와 사업자 정보 필요. 스토어에 회사명 노출.

→ 어느 명의로 낼지(개인/법인)는 발주처와 먼저 합의하세요. 한 번 정하면 변경이 번거롭습니다.

4. 프로젝트를 내 Expo 계정에 연결

STEP 1 — 본인 Expo 계정으로 로그인

```
eas login      # 본인 Expo 계정 이메일/비밀번호  
eas whoami    # 로그인된 계정 확인
```

STEP 2 — 프로젝트 연결 (projectId 자동 주입)

```
eas init      # 본인 계정에 새 프로젝트 생성 + app.json 에 projectId 기록
```

`eas init` 을 실행하면 `app.json` 의 `extra.eas.projectId` 와 `owner` 가 본인 계정 기준으로 채워집니다. 이때부터 푸시 토큰 발급(`getExpoPushTokenAsync`)이 정상 동작합니다.

STEP 3 — 앱 식별자 확인/변경 (선택)

`app.json` 에 이미 설정돼 있습니다. 다른 값으로 출시하려면 여기서 바꾸세요.

```
"ios":      { "bundleIdentifier": "store.shoppinglog.app" }  
"android": { "package":          "store.shoppinglog.app" }
```

⚠ 한 번 스토어에 올린 식별자는 변경 불가능합니다. 출시 전 확정하세요.

5. 푸시 자격증명 설정

Expo Push 서비스가 Apple/Google 의 푸시 게이트웨이로 알림을 전달하려면 자격증명이 필요합니다. 대부분 EAS 가 자동 처리하지만 Android FCM 은 수동 작업이 한 단계 있습니다.

iOS APNs 키

EAS 빌드 시 자동 생성·관리됩니다. `eas build` 중 "푸시 키를 만들까요?" 물으면 **Yes**. 별도 수동 작업 거의 없음. (Apple Developer Program 가입은 필수)

Android FCM (Firebase) — 수동 1회

1 — `console.firebase.google.com` 에서 프로젝트 생성

2 — Android 앱 추가, 패키지명 `store.shoppinglog.app` 입력

3 — 프로젝트 설정 → 서비스 계정 → 새 비공개 키 생성 → JSON 파일 다운로드

4 — 그 JSON 을 EAS 에 등록 (FCM V1)

`eas credentials` # Android → Google Service Account → 위 JSON 업로드

최신 Android 푸시는 **FCM V1 (서비스 계정 JSON)** 방식입니다. 예전 "서버 키" 방식은 폐기됨. 이 JSON 을 EAS 에 등록해야 백엔드가 보낸 푸시가 안드로이드 기기까지 도착합니다.

6. EAS 빌드 — APK / AAB / 개발빌드

`eas.json` 에 3개 프로필이 준비돼 있습니다.

프로필	용도	Android 결과물	iOS 결과물
<code>preview</code>	빠른 실기기 테스트 (사이드로드)	APK	내부배포(.ipa)
<code>production</code>	스토어 제출용	AAB (.aab)	App Store(.ipa)
<code>development</code>	dev client 디버깅(선택)	debug	debug

빌드 명령어

```
# 안드로이드 테스트용 APK (기기에 바로 설치 가능)
eas build --profile preview --platform android

# 스토어 제출용 — 안드로이드 AAB
eas build --profile production --platform android

# 스토어 제출용 — iOS
eas build --profile production --platform ios

# 둘 다 한 번에
eas build --profile production --platform all
```

빌드는 EAS 클라우드에서 진행됩니다. 끝나면 터미널에 **다운로드 링크**가 뜨고, expo.dev → 프로젝트 → Builds 에서도 받을 수 있습니다. AAB 는 이 링크에서 받아 Play Console 에 올립니다.

7. App Store Connect & TestFlight

STEP 1 — App Store Connect 에 앱 생성

appstoreconnect.apple.com → 나의 앱 → + → 신규 앱. 번들ID [store.shoppinglog.app](#) 선택, 앱 이름 입력.

STEP 2 — 빌드 업로드 (자동)

```
eas submit --profile production --platform ios
```

또는 `eas build` 직후 "스토어에 제출할까요?" 에 **Yes**. Apple 로그인 후 자동 업로드됩니다.

STEP 3 — TestFlight 에서 테스트

- App Store Connect → **TestFlight** 탭. 업로드된 빌드가 "처리 중" → 몇 분 후 테스트 가능.
- **내부 테스터**: 같은 팀 멤버(최대 100명) — 심사 없이 즉시 설치.
- **외부 테스터**: 이메일 초대/공개 링크(최대 1만명) — 첫 빌드는 간단한 베타 심사 필요.
- 테스터는 **TestFlight 앱**(앱스토어에서 설치) 으로 초대 수락 후 설치.

TestFlight 빌드는 보통 **90일** 후 만료됩니다. 또 빌드마다 **수출 규정(암호화) 질문**에 답해야 업로드가 끝납니다(대개 "아니오").

8. Android Play Console 내부 테스트

STEP 1 — 앱 생성

play.google.com/console → 앱 만들기. 앱 이름/언어/유형(앱) 설정.

STEP 2 — AAB 업로드

방법 A — 자동 제출:

```
eas submit --profile production --platform android
```

(처음엔 Google 서비스 계정 키 연동이 필요. 수동이 더 쉬우면 방법 B)

방법 B — 수동 업로드: 6번에서 받은 `.aab` 파일을 Play Console **테스트** → **내부 테스트** → **새 버전 만들기** 에서 업로드.

STEP 3 — 내부 테스트 트랙으로 배포

- **내부 테스트** 트랙 권장: 최대 100명, 검토 없이 즉시.
- 테스터 목록(이메일) 등록 → 생성된 **참여 링크(opt-in URL)** 를 테스터에게 전달.
- 테스터가 링크로 옵트인 후 Play 스토어에서 설치.

앱 서명. Play 콘솔의 **Play 앱 서명**을 사용하면 EAS 가 만든 업로드 키로 올리고 구글이 최종 서명합니다. (권장, 기본값) 키를 잃어버려도 복구 가능.

신규 개인 계정 출시 요건. 2023~2024 이후 가입한 **개인** 개발자 계정은, 프로덕션(정식) 출시 전 **비공개 테스트(테스터 12명 이상 · 14일 지속)** 를 요구받을 수 있습니다. **내부 테스트는 영향 없음**이니 먼저 내부 테스트로 동작을 검증하세요.

9. 푸시 알림 실제 동작 테스트

빌드된 앱(또는 dev build)을 **실기기**에 설치한 뒤:

1 — 앱 첫 실행 → 알림 권한 허용

2 — 푸시 토큰 확인

토큰은 코드에서 발급되어 (백엔드 연동 전이면) **콘솔에 출력**됩니다. `npx expo start` 로 dev 실행 시 또는 기기 로 그에서 `ExponentPushToken[...]` 확인.

3 — 테스트 발송

방법 A — Expo Push Tool: `expo.dev/notifications` 에 토큰·제목·본문 넣고 전송.

방법 B — curl:

```
curl -X POST https://exp.host/--api/v2/push/send \
-H "Content-Type: application/json" \
-d '{
  "to": "ExponentPushToken[xxxx]",
  "title": "테스트",
  "body": "푸시 도착 확인",
  "data": { "url": "https://shoppinglog.store/orders" }
}'
```

동작 확인 포인트. ① 알림이 기기에 도착하는가 ② 알림을 **탭하면** 앱이 열리며 `data.url` 주소로 웹뷰가 이동하는가. (앱 이 꺼진 상태에서 탭해도 이동하도록 cold-start 처리까지 구현돼 있음)

10. 자주 막히는 부분 (트러블슈팅)

증상	원인 / 해결
토큰이 <code>null</code> / "projectId 없음" 경고	<code>eas init</code> 안 함 → 4번 실행. 또는 시뮬레이터에서 실행함(실기기 필요)
Expo Go 에서 푸시 안 옴	정상. SDK 53+ 부터 Expo Go 는 원격 푸시 미지원. preview/production 빌드로 테스트
안드로이드만 푸시 안 옴	FCM 서비스계정 JSON 미등록 → 5번. <code>eas credentials</code> 확인
iOS 빌드/제출 실패	Apple Developer Program 미가입 또는 결제 미완료. 멤버십 활성화 확인
Play Console "버전 코드 중복"	같은 versionCode 재업로드. <code>production</code> 프로필이 <code>autoIncrement</code> 라 자동 증가됨 — EAS 로 재빌드
알림 탭해도 이동 안 함	푸시 <code>data</code> 에 <code>url</code> 키가 있어야 함 (백엔드 발송 형식 확인)

11. 명령어 치트시트

```
# 1) 준비
npm install -g eas-cli
npm install
eas login                                # 본인 Expo 계정
eas init                                # projectId 연결

# 2) 점검
npx tsc --noEmit
npx expo lint

# 3) 빌드
eas build --profile preview    --platform android    # 테스트 APK
eas build --profile production --platform android    # 스토어 AAB
eas build --profile production --platform ios         # 스토어 iOS

# 4) 제출
eas submit --profile production --platform ios        # → TestFlight
eas submit --profile production --platform android    # → Play Console

# 5) 자격증명/푸시
eas credentials                                # FCM JSON 등록 등
```

문서 끝. 프론트엔드 코드(Webview + 푸시 프론트)는 구현·검증 완료 상태이며, 빌드/배포 담당자는 위 4→5→6→7/8→9 순서로 진행하면 됩니다. 백엔드 연동은 동봉된 `docs/BACKEND.md` 를 백엔드 담당자에게 전달하세요.